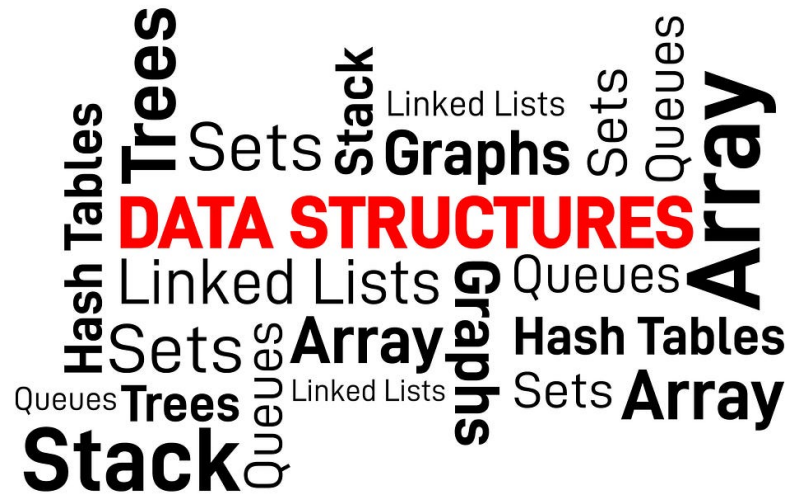
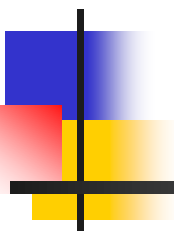




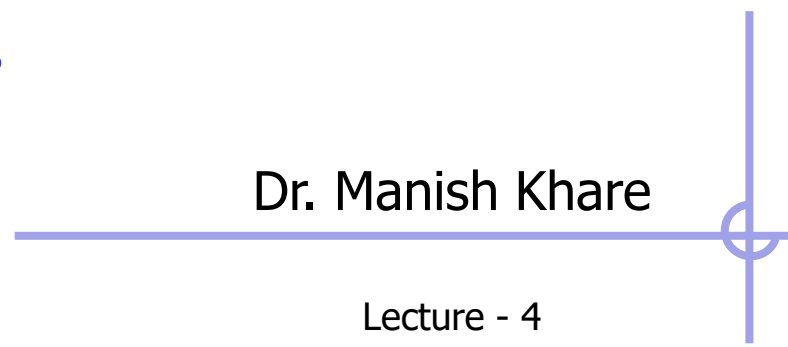
Hash Tables
Trees
Sets
Stack
Linked Lists
Graphs
Sets
Queues
DATA STRUCTURES
Linked Lists
Queues
Sets
Hash Tables
Array
Stack
Queues
Trees
Linked Lists
Graphs
Sets
Array

Data Structures

IT623

Dr. Manish Khare

Lecture - 4

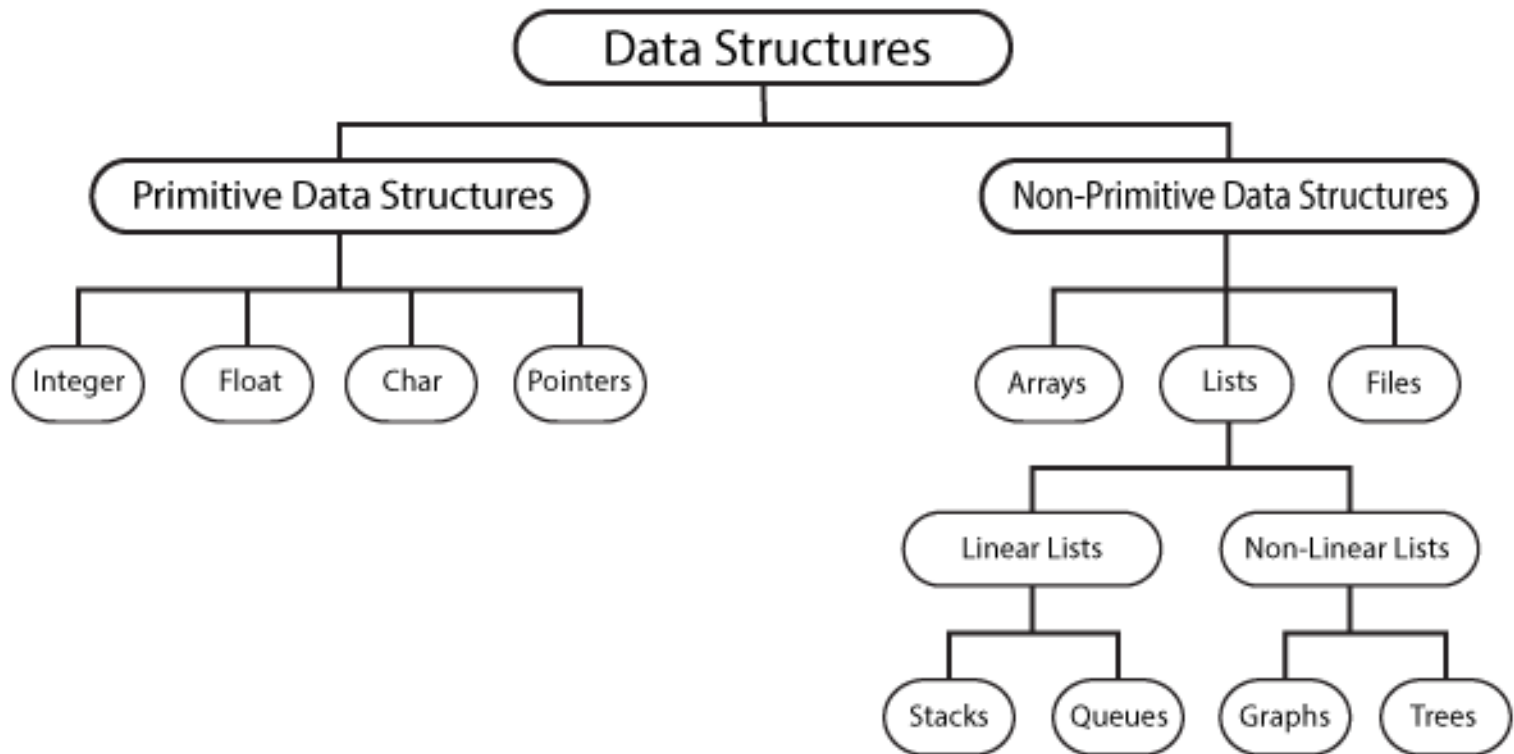


Basic Operations

➤ The data in the data structures are processed by certain operations. The particular data structure chosen largely depends on the frequency of the operation that needs to be performed on the data structure.

- Traversing
- Searching
- Insertion
- Deletion
- Sorting
- Merging

Data Structures



Types Of Data Structure



Array Data Structure

Arrays

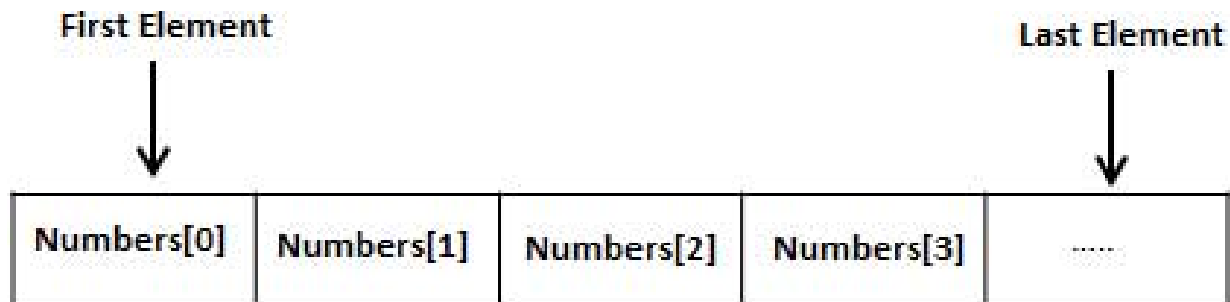


➤ Array is a container which can hold a fix number of items and these items should be of the same type. Most of the data structures make use of arrays to implement their algorithms. Following are the important terms to understand the concept of Array.

- **Element** – Each item stored in an array is called an element.
- **Index** – Each location of an element in an array has a numerical index, which is used to identify the element.

Arrays

- All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.
- Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables. A specific element in an array is accessed by an index.



Arrays Definition

- A set of pairs $\langle \text{index}, \text{value} \rangle$ where for each value of index there is a value from the set item. Index is a finite ordered set of one or more dimensions.
- For example, $\{0, 1, \dots, n-1\}$ for one-dimension, $\{(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)\}$ for two-dimensions, etc.

Array Representation

➤ Functions:

- for all $A \in \text{Array}$, $i \in \text{index}$, $x \in \text{item}$, j , $\text{size} \in \text{integer}$

➤ **Array Create(j, list):** return an array of j dimensions where list is a j -tuple whose i th element is the size of the i th dimension. Items are undefined.

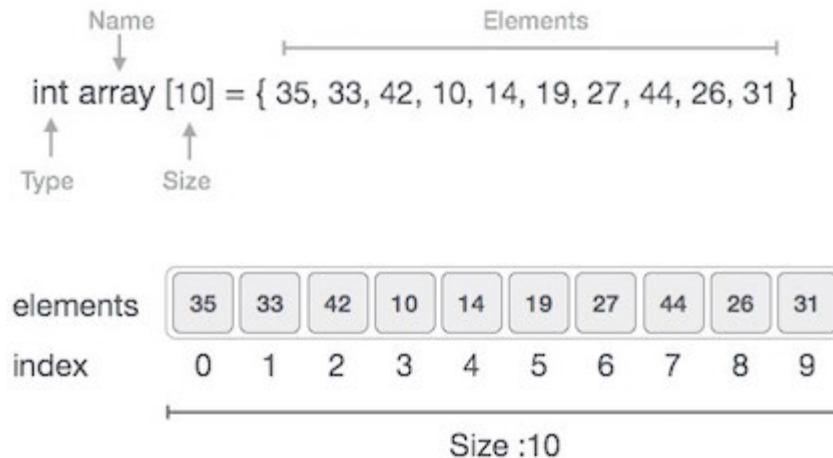
➤ **Item Retrieve (A, i):** if ($i \in \text{index}$) return the item associated with index value i in Array A **else return error**

➤ **Array Store(A, i, x):** if ($i \in \text{index}$)

return an array that is identical to array A except the new pair $\langle i, x \rangle$ has been inserted **else return error**

Arrays Representation

- Arrays can be declared in various ways in different languages. For illustration, let's take C array declaration.



- As per the above illustration, following are the important points to be considered.
- Index starts with 0.
 - Array length is 10 which means it can store 10 elements.
 - Each element can be accessed via its index. For example, we can fetch an element at index 6 as 27.

Arrays Representation

- A two-dimensional array can be considered as a table which will have x number of rows and y number of columns. A two-dimensional array **a**, which contains three rows and four columns can be shown as follows –

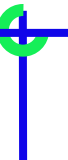
	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

- Thus, every element in the array **a** is identified by an element name of the form `a[ i ][ j ]`, where 'a' is the name of the array, and 'i' and 'j' are the subscripts that uniquely identify each element in 'a'.

Arrays – Basic Operations

➤ Following are the basic operations supported by an array.

- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.



➤ **Traverse Operation**

In traversing operation of an array, each element of an array is accessed exactly for once for processing. This is also called visiting of an array.

➤ **Insertion Operation**

Insert operation is to insert one or more data elements into an array. Based on the requirement, new element can be added at the beginning, end or any given index of array.

➤ **Deletion Operation**

Deletion refers to removing an existing element from the array and re-organizing all elements of an array.

➤ **Search Operation**

You can perform a search for array element based on its value or its index.

➤ **Update Operation**

Update operation refers to updating an existing element from the array at a given index.

Declaring Arrays

- To declare an array in C/C++, a programmer specifies the type of the elements and the number of elements required by an array as follows –
 - `type arrayName [ arraySize ];`
- This is called a *single-dimensional* array. The **arraySize** must be an integer constant greater than zero and **type** can be any valid C data type. For example, to declare a 10-element array called **balance** of type double, use this statement –
 - `double balance[10];`
- Here *balance* is a variable array which is sufficient to hold up to 10 double numbers.

Initializing Arrays

- You can initialize an array in C either one by one or using a single statement as follows –
 - `double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};`
- The number of values between braces { } cannot be larger than the number of elements that we declare for the array between square brackets [].
- If you omit the size of the array, an array just big enough to hold the initialization is created. Therefore, if you write –
 - `double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};`

Accessing Array Elements

- An element is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array. For example –
 - `double salary = balance[9];`

Initializing Two-Dimensional Arrays

➤ Multidimensional arrays may be initialized by specifying bracketed values for each row. Following is an array with 3 rows and each row has 4 columns.

```
■ int a[3][4] = { {0, 1, 2, 3}, /* initializers for row indexed by 0 */  
                 {4, 5, 6, 7}, /* initializers for row indexed by 1 */  
                 {8, 9, 10, 11} /* initializers for row indexed by 2 */ };
```

➤ The nested braces, which indicate the intended row, are optional. The following initialization is equivalent to the previous example –

```
■ int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```


Accessing Two-Dimensional Array Elements

- An element in a two-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array. For example –
 - `int val = a[2][3];`
- The above statement will take the 4th element from the 3rd row of the array.



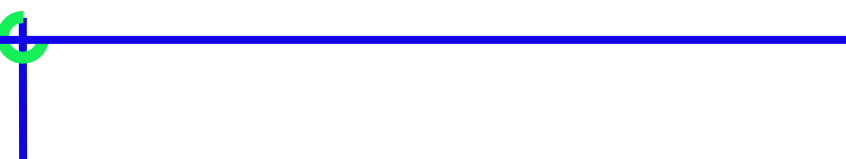
1-Dimensional Array

Insertion Operation in Array

➤ Insert operation is to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array.

➤ Algorithm

- Let **LA** be a Linear Array (unordered) with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm where ITEM is inserted into the K^{th} position of LA –
 - 1. Start
 - 2. Set $J = N$
 - 3. Set $N = N + 1$
 - 4. Repeat steps 5 and 6 while $J \geq K$
 - 5. Set $LA[J + 1] = LA[J]$
 - 6. Set $J = J - 1$
 - 7. Set $LA[K] = \text{ITEM}$
 - 8. Stop



➤ We have learnt how the insertion operation works. It is not always necessary that an element is inserted at the end of an array. Following can be a situation with array insertion –

- Insertion at the beginning of an array
- Insertion at the given index of an array
- Insertion after the given index of an array
- Insertion before the given index of an array

Delete Operation in Array

- Deletion refers to removing an existing element from the array and re-organizing all elements of an array.
- Algorithm
 - Consider **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm to delete an element available at the K^{th} position of LA.
 1. Start
 2. Set $J = K$
 3. Repeat steps 4 and 5 while $J < N$
 4. Set $LA[J] = LA[J + 1]$
 5. Set $J = J + 1$
 6. Set $N = N - 1$
 7. Stop

Search Operation in Array

- You can perform a search for an array element based on its value or its index.
- Algorithm
 - Consider **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm to find an element with a value of **ITEM** using sequential search.
 1. Start
 2. Set $J = 0$
 3. Repeat steps 4 and 5 while $J < N$
 4. IF $LA[J]$ is equal **ITEM** THEN GOTO STEP 6
 5. Set $J = J + 1$
 6. PRINT J, ITEM
 7. Stop

Update Operation in Array

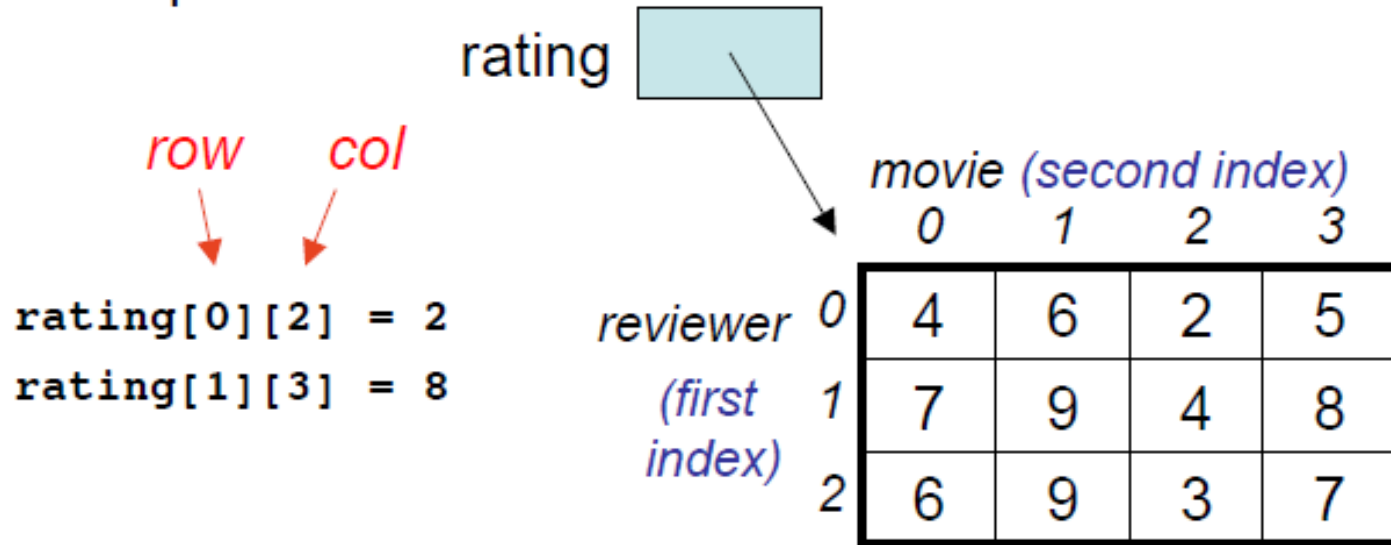
- Update operation refers to updating an existing element from the array at a given index.
- Algorithm
 - Consider **LA** is a linear array with **N** elements and **K** is a positive integer such that $K \leq N$. Following is the algorithm to update an element available at the K^{th} position of LA.
 1. Start
 2. Set $LA[K-1] = \text{ITEM}$
 3. Stop



2-Dimensional Array

2-Dimensional Arrays

- Two-dimensional (2D) arrays are indexed by two subscripts, one for the row and one for the column.
- Example:



Similarity with 1-D Arrays

- Each element in the 2D array must be the same type,
 - either a primitive type or object type.
- Subscripted variables can be used just like a variable:

```
rating[0][3] = 10;
```

- Array indices must be of type `int` and can be a literal, variable, or expression.

```
rating[3][j] = j;
```

- If an array element does not exist, the Java runtime system will give you an

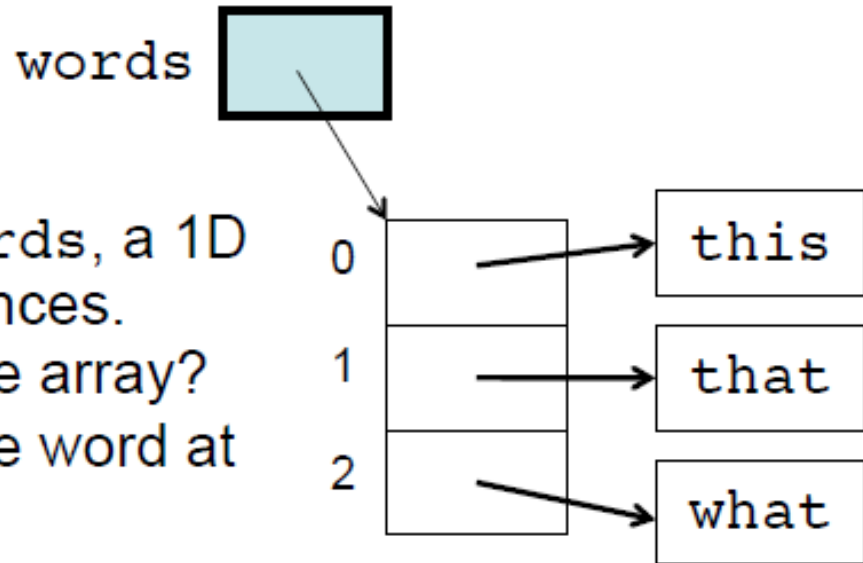
`ArrayIndexOutOfBoundsException`

Size of 2-D Arrays

- When you write a method that has a 2D array as a parameter, how do you determine the size of the array?

Hint:

- Consider a variable `words`, a 1D array of `String` references.
- What is the length of the array?
- What is the length of the word at index 2?



- 
- Addition of two 2-D Array
 - Multiplication of two 2-D Array
 - Traverse a 2-D Array
 - Transpose a 2-D Array

Addition of two 2-D Array

➤ Add ():

- Description: Here A is a two – dimensional array with M rows and N columns and B is a two – dimensional array with X rows and Y columns. This algorithm adds these two arrays.

1. If $(M \neq X)$ or $(N \neq Y)$ Then
2. Print: Addition is not possible.
3. Exit

[End of If]

4. Repeat For $I = 1$ to M
5. Repeat For $J = 1$ to N
6. Set $C[I][J] = A[I][J] + B[I][J]$

[End of Step 5 For Loop]

[End of Step 6 For Loop]

7. Exit



➤ First, we have to check whether the rows of array A are equal to the rows of array B or the columns of array A are equal to the columns of array B. If they are not equal, then addition is not possible and the algorithm exits. But if they are equal, then first for loop iterates to the total number of rows i.e. M and the second for loop iterates to the total number of columns i.e. N. In step 6, the element $A[I][J]$ is added to the element $B[I][J]$ and is stored in $C[I][J]$ by the statement:

- $C[I][J] = A[I][J] + B[I][J]$

Multiplication of two 2-D Array

➤ Multiply ():

- Description: Here A is a two – dimensional array with M rows and N columns and B is a two – dimensional array with X rows and Y columns. This algorithm multiplies these two arrays.

1. If $(M \neq Y)$ or $(N \neq X)$ Then
2. Print: Multiplication is not possible.
3. Else
4. Repeat For I = 1 to N
5. Repeat For J = 1 to X
6. Set $C[I][J] = 0$
7. Repeat For K = 1 to Y
8. Set $C[I][J] = C[I][J] + A[I][K] * B[K][J]$
[End of Step 7 For Loop]
[End of Step 5 For Loop]
[End of Step 4 For Loop]
[End of If]
9. Exit



➤ First we check whether the rows of A are equal to columns of B or the columns of A are equal to rows of B. If they are not equal, then multiplication is not possible. But, if they are equal, the first for loop iterates to total number of columns of A i.e. N and the second for loop iterates to the total number of rows of B i.e. X. In step 6, all the elements of C are set to zero. Then the third for loop iterates to total number of columns of B i.e. Y. In step 8, the element $A[I][K]$ is multiplied with $B[K][J]$ and added to $C[I][J]$ and the result is assigned to $C[I][J]$ by the statement:

- $C[I][J] = C[I][J] + A[I][K] * B[K][J]$

Traverse a 2-D Array

➤ Traverse ():

➤ Description: Here A is a two – dimensional array with M rows and N columns. This algorithm traverses array A and applies the operation PROCESS to each element of the array.

1. Repeat For $I = 1$ to M
2. Repeat For $J = 1$ to N
3. Apply PROCESS to $A[I][J]$
[End of Step 2 For Loop]
[End of Step 1 For Loop]
4. Exit

- 
- The first for loop iterates from 1 to M i.e. to the total number of rows and the second for loop iterates from 1 to N i.e. to the total number of columns. In step 3, it applies the operation **PROCESS** to the elements of the array A .

Transpose a 2-D Array

➤ Transpose ():

➤ Description: Here A is a two – dimensional array with M rows and N columns. This algorithm transposes the array.

1. Repeat For $I = 1$ to M

2. Repeat For $J = 1$ to N

3. Set $B[J][I] = A[I][J]$

[End of Step 2 For Loop]

[End of Step 1 For Loop]

4. Exit

- 
- The first for loop iterates from 1 to M i.e. total number of rows and second for loop iterates from 1 to N i.e. total number of columns. In step 3, the element at location $A[I][J]$ is assigned to $B[J][I]$ by the statement $B[J][I] = A[I][J]$.